High-Efficiency Exam Notes

1) DOM (Document Object Model)

Definition

- DOM is an **object-oriented representation of an HTML document**.
- It represents the page as a **tree structure**.
- It is **not a JavaScript feature itself**.
- It is a **Web API**.
- JavaScript is the most common language used to manipulate it, but other languages can also use it.

DOM tree idea

- document
  - <html>
    - <head>
      - <title>
    - <body>
      - headings, links, paragraphs, etc.

Why it matters

- It lets programs **read, change, add, or remove** HTML elements.
- It is the bridge between the webpage structure and scripting.
  Slides at the beginning of the PDF explain DOM as a tree and as a Web API.

2) Evolution of Frontend Development

Simple evolution

- Earlier: app code directly worked with **HTML + CSS + JS + DOM + Web APIs**
- Next: **jQuery** helped simplify DOM work
- Modern: frameworks/libraries handle much of the DOM work for us

Main idea

- Frontend development moved from **manual DOM manipulation** to **structured, component-based development**.
- This improves **speed, maintainability, and scalability**.
  The frontend evolution diagram and React introduction slides show this shift from direct DOM handling to library/framework-based development.

3) React JS

Definition

- React is a **JavaScript library for building user interfaces**.
- It was developed and maintained by **Facebook and Instagram**.
- In MVC terms, React mainly serves as the **View**.

Why React is used

- Good for **large web applications**

- Handles **changing data over time**
- Updates UI **without reloading the full page**
- Focuses on **speed, simplicity, and scalability**

Related

- **React Native** -> mobile apps
- **React VR** -> VR-related apps

  These React basics are presented in the React JS slides near the start of the deck.

4) React Components

Definition

- Components are one of the **core concepts** in React.
- They are the **building blocks** of the UI.

Key ideas

- UI is built from **small pieces**
- Examples: buttons, text areas, cards, images
- Components can be **reused**
- Components can be **nested**

Why important

- Makes code easier to:
  - reuse
  - manage
  - test
  - scale

Easy memory line

- **React app = UI made from reusable Lego blocks**.

  The components slide explains that everything on screen can be broken into reusable, nestable components.

5) Notable React Features

a) One-Way Data Flow

- Data flows in **one direction**
- Usually from **parent to child**
- There is a **single source of truth**
- This makes state easier to track and debug

b) Virtual DOM

- React does not directly update the real DOM every time
- It creates a **virtual DOM**
- Then it compares changes and updates **only what changed**
- This improves performance

c) JSX

- JSX is the syntax used in React to describe UI
- It looks like **HTML/XML inside JavaScript**
- Makes UI code easier to read and write
- React safely escapes values by default, which helps reduce some XSS risks

Easy memory line

- **React = One-way data + Virtual DOM + JSX**.
  These three features are highlighted together in the “Notable Features” slides.

6) React Project Structure

A React project is usually organized into folders such as:

- **components** -> reusable UI parts
- **layouts/pages/views** -> screen-level UI
- **services** -> API calls / side effects
- **controllers/store** -> state or app data flow
- **utils** -> helper functions
- **hooks** -> custom hooks
- **public** -> static assets
- **src** -> main source code

Main idea

- Good folder structure makes a project easier to read and maintain.
  The project structure slide shows a typical React app organization with UI, services, helpers, hooks, and app entry points.

7) State in Web Apps

Definition

- State means the **current data or condition** of an application at a specific time.

Why important

- State affects:
  - what the user sees
  - how the app behaves
  - how components respond to actions

Examples

- Logged in / logged out
- Counter value
- Form input text
- Selected item
- Theme mode
  The state slides define state as the current condition/data of the app and explain why it affects UI behavior and appearance.

8) State Management in Web Apps

There are two main types:

a) Local State Management

- State belongs to one component or a small area
- Managed inside that component

b) Global State Management

- State is shared across many parts of the app
- Often managed using **Redux**

Simple flow

- User action -> state update -> UI re-renders
  The state management slides distinguish local and global state and include Redux as the global option.

9) Event Handlers

Definition

- Event handlers are functions that run in response to user actions.

Examples of actions

- click
- key press
- form submit
- mouse move

In React

- Usually written as:
  - methods in class components, or
  - arrow functions in functional components
- In JSX, event names use:
  - onClick
  - onChange
  - onSubmit

Event object

- Event handlers can receive an **event object**
- It contains information like:
  - target element
  - pressed key
  - mouse details
    The event handlers slide explains React event handlers, JSX naming with on..., and the event object.

10) Bundling

Definition

- Bundling means combining multiple files into fewer files or one bundle.

Usually bundled

- JavaScript
- CSS
- assets

Why bundling is useful

- Reduces number of HTTP requests
- Improves loading performance
- Reduces latency
- Tools analyze dependencies and create optimized bundles

Examples of bundlers/tools

- Webpack
- Parcel
- Vite

  The bundling section explains that combining files reduces requests and improves performance.

11) Webpack

Definition

- Webpack is an **open-source module bundler**.

What it does

- Bundles JS and assets like:
  - CSS
  - images
  - fonts
- Analyzes dependency relationships
- Produces an optimized bundle

Features

- development server
- live changes preview
- code splitting
- lazy loading
- tree shaking
- hot module replacement

Key point

- Very powerful and highly configurable.

  The Webpack slide lists its role as a bundler plus features such as code splitting, lazy loading, and HMR.

12) Parcel

Definition

- Parcel is also a web bundler/build tool.

Why people like it

- **Zero-config**
- Very easy to start
- Handles many file transformations automatically
- Can run builds in parallel
- Includes a dev server

Key point

- Easier and simpler than Webpack for many use cases.
  The Parcel slide emphasizes zero-configuration, automatic transforms, and ease of use.

13) Vite

Definition

- Vite is a **modern build tool and development server**.

Key features

- Very fast startup
- Uses modern browser features like **native ES modules**
- Quickly serves app code during development
- Supports code splitting and tree shaking for production builds

Key point

- Popular for modern React and Vue front-end development because it is **fast**.
  The Vite slide highlights fast startup, native ES modules, and efficient front-end development.

14) Redux

Definition

- Redux is a **pattern and library** for managing application state.

Use

- Best for **global state**
- Used when many parts of the app need the same data

Main idea

- Keeps state in a **centralized store**
- State updates happen in a **predictable** way

Redux one-way flow

- **Action -> State -> View**

Why useful

- Better control of large app state

- Easier to debug and track changes
  The Redux slides describe it as centralized global state management with predictable updates and one-way flow.

15) React Hooks

Definition

- Hooks let functional components use React features like state and side effects.

Common hooks

useState

- Adds state to a functional component
- Returns:
  - current state value
  - function to update it

useEffect

- Used for side effects
- Examples:
  - fetching data
  - updating DOM
  - subscriptions

useContext

- Accesses values from React Context

useReducer

- Alternative to useState
- Good for more complex state logic

Rules of Hooks

- Use hooks at the **top level**
- Do **not** use them inside loops or conditions

Easy memory line

- **State, Effect, Context, Reducer** = most important exam hooks.
  The hooks slide lists these hooks, their purposes, and the rule that hooks should be used only at the top level.

16) Managing State / Reacting to Inputs

Main ideas

- As apps grow, state organization becomes very important
- Bad state design causes bugs
- Duplicate or redundant state is a common source of errors
- React is **declarative**
- Instead of manually changing each UI piece, you describe how UI should look in each state

Key exam phrase

- React does not focus on **manual UI manipulation**
- React focuses on **describing UI based on state**
  The "Managing State | Reacting to Inputs" slide explains React's declarative approach and why poor state design causes bugs.

17) Good State Structure Rules

Group related state

- If values always change together, keep them together

Avoid contradictions in state

- State values should not disagree with each other

Avoid redundant state

- Do not store what can be calculated from other values

Avoid duplication in state

- Keep one reliable source instead of copying the same data in many places

Avoid deeply nested state

- Deep nesting is hard to update
- Prefer flatter state when possible

Easy memory line

- **Group, avoid contradiction, avoid redundancy, avoid duplication, avoid deep nesting**.
  These rules are directly summarized in the "Managing State | State Structure" slide.

18) State Sharing / Lifting State Up

When needed

- When two components need to stay synchronized

Solution

- Remove state from both components
- Move it to their **closest common parent**
- Pass it down through **props**

Name of this concept

- **Lifting state up**

Why important

- Very common in React apps
  The state sharing slide explains lifting state up as moving shared state to the nearest parent and passing it down via props.

19) Preserving and Resetting State

Main idea

- State is isolated between components
- React tracks state based on a component's **position in the UI tree**

Important

- If component identity/position changes, state may reset
- If the same component remains in the same place, state is usually preserved

Exam point

- State is connected to **where the component sits in the tree**.
  The preserving/resetting slide explains that React ties state to component position in the UI tree.

20) Context

Problem

- Passing data through many levels using props can become long and messy

Solution

- **Context**

What Context does

- Lets a parent make data available to deeply nested components
- Avoids passing props through every intermediate component

Common use cases

- theme
- logged-in user
- language
- shared settings

Easy memory line

- **Props for nearby sharing, Context for deep sharing**.
  The context slide describes prop drilling and shows Context as the way to pass data deep into the tree.

21) Web Application Security

OWASP

- OWASP = **Open Web Application Security Project**
- It is a nonprofit organization focused on software security

OWASP Top 10

- A list of the top common web application security risks
  The last slides introduce OWASP and the OWASP Top 10 list.

22) OWASP Top 10 (2021)

1. **Broken Access Control**
   Users can access things they should not.
2. **Cryptographic Failures**
   Sensitive data is exposed because encryption is weak or missing.
3. **Injection**
   Malicious input is interpreted as commands or queries.

4. **Insecure Design**
   Security was not properly considered in the system design.
5. **Security Misconfiguration**
   Unsafe settings, default configs, or poor setup create vulnerabilities.
6. **Vulnerable and Outdated Components**
   Old/insecure libraries and frameworks create risk.
7. **Identification and Authentication Failures**
   Weak login, session, or identity controls.
8. **Software and Data Integrity Failures**
   Untrusted software updates, plugins, or data can be harmful.
9. **Security Logging and Monitoring Failures**
   Attacks are not properly detected or tracked.
10. **Server-Side Request Forgery (SSRF)**
    The server is tricked into making requests it should not make.
    The final slide lists the OWASP Top 10 vulnerabilities for 2021.

Super-Fast Last-Minute Revision Sheet

1-line memory answers

- **DOM** = tree-like object representation of HTML, exposed as a Web API.
- **React** = JavaScript library for building UI.
- **Component** = reusable UI building block.
- **One-way data flow** = parent to child data movement.
- **Virtual DOM** = React compares a virtual tree and updates only changed parts.
- **JSX** = HTML-like syntax inside JavaScript for UI.
- **State** = current data/condition of the app.
- **Local state** = used inside one component.
- **Global state** = shared across many parts; often managed with Redux.
- **Event handler** = function triggered by user actions.
- **Bundling** = combining files to improve performance.
- **Webpack** = powerful configurable bundler.
- **Parcel** = easy zero-config bundler.
- **Vite** = very fast modern dev tool/build tool.
- **Redux** = centralized predictable state management.
- **Hooks** = React features for functional components.
- **useState** = local state.
- **useEffect** = side effects.
- **useContext** = access context.
- **useReducer** = complex state logic.
- **Lifting state up** = move shared state to common parent.

- **Context** = avoid prop drilling.
- **OWASP** = major web security organization.
- **OWASP Top 10** = top common web security risks.
  These are condensed from the full topic flow across the slide deck.